

**A Probabilistic Knowledge System for Breast Cancer prediction based on
Bayesian Networks and Machine Learning Technology**

by

Metaxioti Maria

A THESIS

submitted in partial fulfillment of the requirements for the degree

MASTER OF SCIENCE

DEPARTMENT OF INFORMATICS ENGINEERING

SCHOOL OF ENGINEERING

HELLENIC MEDITERRANEAN UNIVERSITY

December 2024

Approved by:
Supervisor
Emmanouil Marakakis, Emeritus Professor

Abstract

Breast cancer is a serious health issue, particularly among women. Early diagnosis is crucial as it significantly increases the chances of successful treatment. Recent advances in machine learning have demonstrated high accuracy in predicting breast cancer through numerous algorithms. Although these algorithms achieve high accuracy, most of them lack interpretability and explainability which are essential for understanding the decision-making process. Additionally, these algorithms often lack web-based accessible interfaces for users, making them challenging to interact with.

This study presents the development of a high accuracy breast cancer diagnostic system using Bayesian networks. The system includes a web application designed for both ordinary users, i.e. potential patients, and the knowledge engineer. For ordinary users, the system allows them to input the necessary information and receive a diagnostic result based on the provided data. The knowledge engineer, through a specialized interface, can trigger the three modules of machine learning process, that is data preprocessing, creation of the model and model evaluation.

The proposed solution promotes accessibility in breast cancer diagnosis. It improves accessibility of health services through web-based applications. This is of great help in the field of medicine due to efficient support to potential patients. Moreover, this study highlights on the importance of interpretability of the derived model and on the explainability of the diagnostic system. These characteristics have high impact in the use of machine learning technology particularly in medicine because they build trust.

Abstract

Ο καρκίνος του μαστού αποτελεί σοβαρό ζήτημα υγείας, ιδιαίτερα μεταξύ των γυναικών. Η έγκαιρη διάγνωση είναι κρίσιμη, καθώς αυξάνει σημαντικά τις πιθανότητες επιτυχούς θεραπείας. Οι πρόσφατες εξελίξεις στη μηχανική μάθηση έχουν δείξει υψηλή ακρίβεια στην πρόβλεψη του καρκίνου του μαστού μέσω διαφόρων αλγορίθμων. Αν και οι αλγόριθμοι αυτοί επιτυγχάνουν υψηλή ακρίβεια, οι περισσότεροι στερούνται ερμηνευσιμότητας και εξηγησιμότητας, που είναι απαραίτητες για την κατανόηση της διαδικασίας λήψης αποφάσεων. Επιπλέον, συχνά δεν διαθέτουν διαδικτυακές προσβάσιμες διεπαφές για τους χρήστες, καθιστώντας δύσκολη την αλληλεπίδραση μαζί τους.

Η παρούσα μελέτη παρουσιάζει την ανάπτυξη ενός συστήματος υψηλής ακρίβειας για τη διάγνωση του καρκίνου του μαστού χρησιμοποιώντας Δίκτυα Bayes. Το σύστημα περιλαμβάνει μια διαδικτυακή εφαρμογή σχεδιασμένη τόσο για απλούς χρήστες, δηλαδή πιθανούς ασθενείς, όσο και για τον μηχανικό γνώσης. Για τους απλούς χρήστες, το σύστημα τους επιτρέπει να εισάγουν τις απαραίτητες πληροφορίες και να λαμβάνουν ένα διαγνωστικό αποτέλεσμα βάσει των δεδομένων που παρέχουν. Ο μηχανικός γνώσης, μέσω μιας εξειδικευμένης διεπαφής, μπορεί να ενεργοποιεί τις τρεις ενότητες της διαδικασίας μηχανικής μάθησης, δηλαδή την προεπεξεργασία δεδομένων, τη δημιουργία του μοντέλου και την αξιολόγηση του μοντέλου.

Η προτεινόμενη λύση προάγει την προσβασιμότητα στη διάγνωση του καρκίνου του μαστού. Βελτιώνει την προσβασιμότητα στις υπηρεσίες υγείας μέσω διαδικτυακών εφαρμογών, κάτι που είναι ιδιαίτερα χρήσιμο στον τομέα της ιατρικής, παρέχοντας αποτελεσματική υποστήριξη στους πιθανούς ασθενείς. Επιπλέον, η μελέτη αυτή δίνει έμφαση στη σημασία της ερμηνευσιμότητας του προκύπτοντος μοντέλου και της εξηγησιμότητας του διαγνωστικού συστήματος. Αυτά τα χαρακτηριστικά έχουν μεγάλο αντίκτυπο στη χρήση της τεχνολογίας μηχανικής μάθησης, ιδιαίτερα στην ιατρική, καθώς ενισχύουν την εμπιστοσύνη.

Acknowledgements

I would like to acknowledge and deeply thank my supervisor, Dr. Emmanouil Marakakis, whose guidance and advice carried me through all the stages of writing my thesis. I would also like to thank my committee members for generously providing their time and attention for the defense of this thesis.

I would also like to give special thanks to the people closest to me, Ilias, Antonis, Alexia, my family, as well as my friends, for their continuous support and understanding throughout the writing of this thesis, as well as my life, in general.

Table of Contents

Abstract	3
Abstract	4
Acknowledgements	5
Table of Contents	6
List of Figures	7
List of Tables	8
1. Introduction	9
1.1 Overview and Motivation	9
1.2 Thesis Structure	9
2 Background	10
2.1 Machine Learning	10
2.2 Probabilistic Logic Reasoning	11
2.3 Bayesian Networks	12
2.4 EM Algorithm	13
2.5 Evaluation of Trained Models	13
2.5.1 Accuracy	14
2.5.2 Precision	14
2.5.3 Sensitivity (Recall)	14
2.5.4 Specificity	14
2.5.5 Receiver Operating Characteristic (ROC) Curve	15
2.5.6 F1 Measure	15
2.6 Development tools	15
2.6.1 Python	15
2.6.2 ProbLog	16
2.6.3 React	17
2.6.4 Flask	17
3 Related Work	18
4 Methodology	20
4.1 The Dataset	20
4.2 The Bayesian Network	20
4.3 The Architecture of the Machine Learning System	21
4.4 Preprocessing	23
4.5 Model Construction	24
4.6 Training	27
4.7 Testing	29
4.8 Application Phase	29
4.8.1 Front-End	30
4.8.2 Back-End	32
5 Results and Discussion	33
6 Conclusion and Future Work	36
7 Bibliography	37

List of Figures

Figure 1: Bayesian Network with joint probability distribution	13
Figure 2: CoinFlip ProbLog programme.....	16
Figure 3: Bayesian Network Diagram.....	21
Figure 4: Machine Learning Phase - System Architecture Diagram	22
Figure 5: Application Phase - System Architecture Diagram	23
Figure 6: Sample of the first 10 instances of the cleaned dataset using the first approach.	24
Figure 7: Histogram showing the ranges of the values inside the Dataset.....	25
Figure 8: Dataset record	25
Figure 9: The untrained model that has been incorporated to the system.	27
Figure 10: Sample of Trained Model. (1/2)	28
Figure 11: Sample of Trained Model. (2/2)	28
Figure 12: Sample of Training Evidence.	28
Figure 13: Application's Home Page.....	30
Figure 14: Knowledge Engineer Menu	31
Figure 15: User Page	31
Figure 16: System results example.....	32
Figure 17: The model's ROC Curve	34
Figure 18: The model's Confusion Matrix.....	34

List of Tables

Table 1: Model Performance	33
Table 2: The model's Confusion Matrix	35

1. Introduction

1.1 Overview and Motivation

Breast cancer is one of the most prevalent and life-threatening cancers affecting women globally. Accurate and early diagnosis is crucial to enhance patient outcomes, as it allows prompt treatment and more effective disease management. Given the rise of data science and machine learning, there is a significant opportunity to enhance the efficiency and accuracy of breast cancer diagnosis through automated systems. This master thesis presents a system for Breast Cancer diagnosis with probabilistic logical reasoning using Bayesian Networks. Bayesian networks, with their ability to model probabilistic relationships and handle uncertainty, offer a promising approach for medical diagnosis. While other machine learning algorithms may achieve greater accuracy, this thesis aims to signify the importance of the interpretability and explainability features of our system. The interpretability of the derived models is due to the Bayesian Networks modelling. Moreover, the constructed diagnostic system can explain its reasoning to the decision it has derived.

The Breast Cancer Wisconsin (Diagnostic) Data Set [1] is used for the creation of a diagnostic model. The elements in this dataset are computed from a computerized picture of a breast mass's fine needle aspiration (FNA). These elements describe the attributes of the cell nuclei that appears in the image. The selection of this data set was based on its wide acceptance among the research community, as related work suggests, providing a solid foundation for this thesis's results. For the development of this thesis the probabilistic logic programming language ProbLog is used. ProbLog is a stand-alone language but can also be found as a library in Python [2]. The Python programming language will also be used for the preprocessing of the data, as well as the Javascript Framework, React, which will be used for the creation of the user interface. Lastly, the Python Framework, Flask, that will be used to handle HTTP requests. Lastly, the EM Algorithm is used to estimate the parameters of the derived model. There is also an overall evaluation of our system performance as well as its ROC curve analysis.

1.2 Thesis Structure

The organization of this thesis is in six chapters. The background necessary to understand the concepts and technologies used in this study is provided in Chapter 2, including an overview of Bayesian networks and their application in machine learning. Chapter 3 reviews related work in the field of breast cancer diagnosis using machine learning methodologies, highlighting previous studies, and findings. Chapter 4 describes the methodology employed in this study, presenting in details the processes of data preprocessing, model construction, system design and construction. Chapter 5 presents the results of the model evaluation, discussing the performance metrics, and interpreting the findings. Finally, Chapter 6 concludes the thesis by summarizing the key insights, discussing the limitations of the study, and suggesting directions for future research.

2 Background

This chapter provides the foundational knowledge necessary to understand the methodologies and tools utilized in this thesis. It begins with an overview of machine learning, explaining its significance and applications in various domains. This is followed by a discussion on probabilistic logic reasoning, emphasizing its role in decision-making processes. This chapter then delves into the Bayes Theorem, a fundamental concept underpinning Bayesian networks. The Expectation-Maximization (EM) algorithm is also covered, detailing its use in parameter estimation for probabilistic models. Furthermore, the chapter introduces the programming languages and frameworks employed in this study, including Python for its versatility in data analysis, ProbLog for probabilistic logic programming, and finally React and Flask for developing the front-end and back-end components of the application, respectively.

2.1 Machine Learning

Machine Learning (ML) is rooted in the convergence of computer science, statistics, artificial intelligence and it enables systems to learn from data and improve their performance over time without the need for explicit programming [3]. It is a popular and highly researched field in computer science. According to Tom Mitchel, a computer program is said to learn from experience with respect to some class of tasks and the performance measured also improves with experience [4]. Experience occurs in computer systems in the form of data, and the primary objective of machine learning is to construct learning algorithms that generate models from data. Machine learning involves programming computers to optimize a performance metric using example data or prior experiences. A model is specified with certain parameters, and learning entails running a computer program to refine these parameters based on training data or past experiences. The model can serve a predictive purpose to forecast future outcomes, a descriptive purpose to extract insights from data, or both. [5]. Overall, ML is the study of *learning algorithms* if we take computer science to be the domain of *algorithms* [6].

In order to perform ML, firstly, there must be available some *data*. A *data set* is an organized collection of data. They usually address one subject at a time and are typically linked to a distinct body of work. A data set's information components are related to one other. Data sets can be used by experts in scientific and statistical fields, such as biologists, to examine information on a region's climate or environment. The ML process also needs an *untrained model*. An untrained model is a machine learning model that has been created but hasn't yet been exposed to any data for learning. At this point, the model is aware of the types and structures of data it will use, but it is unaware of any patterns or connections found in the data. In essence, it is a blank slate that requires dataset training in order to acquire predictive capabilities. *Learning* or *training* refers to the process of utilizing machine learning algorithms to generate *trained models* from data. The data utilized in the training phase is referred to as *training data set*, with each record serving as a training example and the collection of all training examples makes the *training set*.

Consider, for example, a decision tree model designed to predict if a tumor is benign or malignant depending on features like size, shape, and texture. The untrained decision tree has nodes and branches prior to training, but it is unaware of the relationship between these features and tumor classification. The model can only become capable of making predictions after being fed a labeled dataset and allowed to "learn" from the data. *Model evaluation or testing phase* is the process of evaluating the performance of the derived model with a sample of data called *test data set* which is disjoint from the trained data set. It is desirable both data sets, i.e. the

training data set and the test data set, to be from the same collection of data. Learning issues can essentially be divided into two types based on whether the training data is labeled or unlabeled: *supervised learning* (for example, *regression and classification*) and *unsupervised learning* (for example, *clustering*) [6].

The two main categories of *Supervised Learning* are the following.

- **Classification:** A classification problem is one in which the prediction output is discrete, such as “sick” and “not sick”.
- **Regression:** A regression problem occurs when the forecast output is continuous, such as a real number.

The two main categories of *Unsupervised Learning* are the following.

- **Clustering:** Clustering is a way of learning through the discovery of intrinsic groupings in data, such as grouping fruits based on colour, etc.
- **Rule-Based:** Rule-based learning is a method that incorporates explicit rules or heuristics into the learning process. It blends the adaptability of unsupervised learning with the transparency of rule-based systems, allowing for more interpretable and adaptable discovery of valuable insights.

2.2 Probabilistic Logic Reasoning

Probabilistic logic reasoning is an area within artificial intelligence that makes use of ideas from probability theory and logic to represent and reason about uncertainty in an organized and codified way [7]. Unlike classical logic, which works with deterministic relationships and precise conclusions, probabilistic logic reasoning accepts and accounts for the uncertainty inherent in real-world facts and knowledge [8][9].

At its foundation, probabilistic logic reasoning aims to answer questions such as: How likely is a specific event to occur given certain information or circumstances? What is the most likely explanation for a series of observed events? These questions are particularly relevant in domains where uncertainty is prevalent, such as decision-making under uncertainty, pattern recognition, and diagnostic reasoning [10].

Bayesian and Markov logic networks are the two main frameworks for probabilistic logic reasoning.

1. **Bayesian networks (BNs)** are graphical models that use a directed acyclic graph to express probability correlations between variables [11], [12]. Each node in the network represents a random variable, while the edges connecting them indicate probabilistic dependencies [13], [14]. Bayesian networks, which give a coherent framework for reasoning under uncertainty, are commonly employed for tasks such as probabilistic inference, anomaly detection, and decision support systems [5].
2. **Markov Logic Networks (MLNs)** are a combination of first-order logic and Markov networks that represent uncertain knowledge and conduct probabilistic inference [15]. In MLNs, logical formulas are assigned weights that reflect the strength of the relationship between predicates. These weights are used to build a probability distribution over potential worlds, which enables probabilistic reasoning and inference. MLNs provide a versatile and

expressive framework for modeling complicated relational data, with applications in natural language processing, social network analysis, and bioinformatics.

Probabilistic logic reasoning has a wide range of applications, including:

- **Medical Diagnosis:** Bayesian networks can be used to simulate the probabilistic links between symptoms and diseases, making it easier to diagnose and treat medical disorders.
- **Information Retrieval:** Markov logic networks can be used for information retrieval tasks like document ranking and relevance evaluation by using probabilistic reasoning to improve search results.
- **Robotics:** Probabilistic logic reasoning approaches are employed in robotics for tasks like localization, mapping, and path planning, when sensor measurements and environmental observations contain uncertainty.

As the volume of data generated grows exponentially, probabilistic logic reasoning offers a powerful framework for reasoning under uncertainty and making informed judgments in complicated and unpredictable contexts.

2.3 Bayesian Networks

Bayesian Networks are based on Bayes' Law, named after Thomas Bayes. Bayes theorem is a fundamental theorem in probability theory that defines how to update the probability of a hypothesis in the presence of new information. This was one of the first approaches which was used in expert systems and in knowledge-based system in general from their early development since 1980 [16], [17]. The introduction of probability theory and graphical models in machine learning contributed to the development of Machine learning. Moreover, Bayesian networks were further developed for representation and reasoning in Machine learning [11], [12]. In general, Bayesian networks offer a logical framework for reasoning under uncertainty and is widely applied in domains such as statistics, machine learning, and artificial intelligence [18], [19], [20].

In Machine learning, the Bayesian networks are used a lot because the graphical model of a Bayesian network expresses independence of the nodes of the network. That is, given its parents, every node is independent of its predecessors. In addition, each node of a Bayesian network that is corresponding to a random variable, is formally expressed as random variables. We are usually interested in evaluating the joint probability distribution of the network because from the joint probability distribution we can evaluate the probability of any node in the network. These features of Bayesian networks are very important in machine learning. The joint probability distribution of nodes of a Bayesian network can be expressed in a succinct way which has an efficient evaluation. Let's assume that we can model our problem using Bayesian networks and we have a data set in tabular form where each column corresponds to a feature of our model and the column has the data values of that feature. That is, a column is a node in the graphical model and a random variable in the probabilistic model. Each row of the tabular data set has the values of a particular record of the data set. The probability of each row is joint probability distribution for these specific values of the row.

Below (Figure 1) is an example usually used to discuss Bayesian Networks [8], [21].

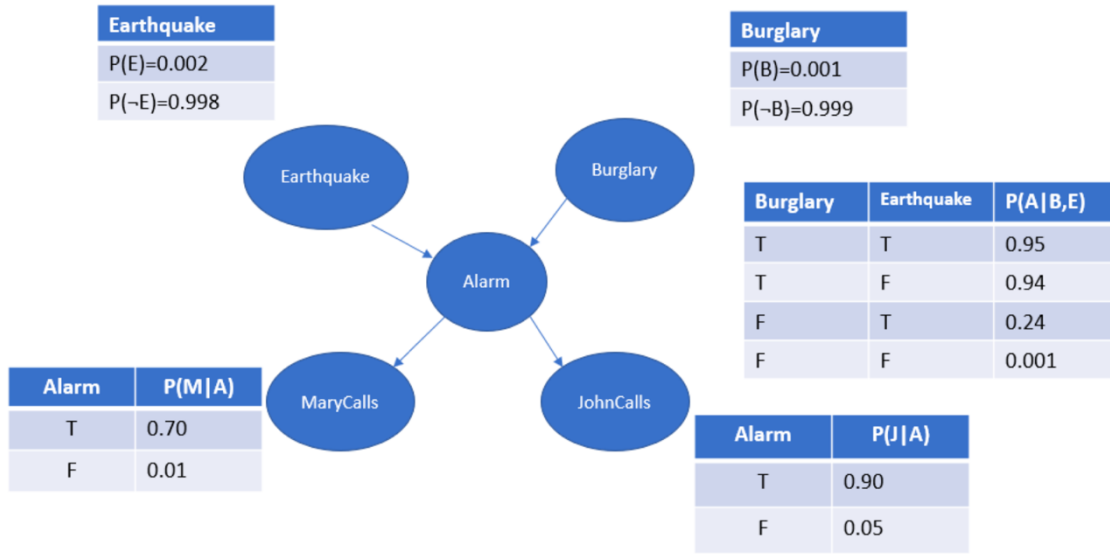


Figure 1: Bayesian Network with joint probability distribution

2.4 EM Algorithm

The most popular algorithm for deriving the training model from a data set in a problem which is modelled as a Bayesian network is the Expectation-Maximization (EM) algorithm. An outline of this algorithm is presented in this subsection.

The Expectation-Maximization algorithm is a strong iterative approach for calculating maximum likelihood estimates (MLE) or maximum a posteriori (MAP) of parameters in statistical models containing latent variables. It is especially beneficial when dealing with incomplete or missing data, in which some variables are either unobserved or partially observed [22].

The EM method includes two major steps: the Expectation step (E-step) and the Maximization step (M-step), which are iteratively repeated until convergence.

1. **Expectation (E-step):** During the E-step, the method computes the expected value of the complete-data log-likelihood function based on the current parameter estimates. This involves determining the values of the latent variables using the observed data and the current estimates. At this stage, the "responsibilities" of each latent variable for generating the observed data are calculated.
2. **Maximization (M-Step):** In order to maximize the expected log-likelihood calculated in the previous step, the method modifies the model parameters during the M-step. This involves finding new parameter estimates that enhance the model's alignment with the observed data. The M-step requires solving a maximization problem to identify the parameters that best account for the observed data, given the estimated latent variables.

2.5 Evaluation of Trained Models

This section examines how well the trained models predict breast cancer. Several criteria are utilized for evaluation, such as precision, specificity, sensitivity, accuracy, area under the ROC curve (AUC) and F1 score. These metrics provide information about the models' reliability,

how effectively they detect cancer, and how frequently they make errors. Each metric is described in length to emphasize its significance in making correct predictions.

2.5.1 Accuracy

Accuracy is defined as the proportion of accurately predicted instances (including true positives and true negatives) to the total number of instances. In medical diagnosis, it indicates how frequently the model generates accurate predictions overall.

$$Accuracy = \frac{True\ Positives\ (TP) + True\ Negatives\ (TN)}{Total\ Population\ (TP + TN + FP + FN)}$$

While accuracy provides a broad picture of the model's performance, it may not be the optimal metric in circumstances when the dataset is imbalanced, such as in medical diagnosis (where there are disproportionately more healthy cases than unhealthy ones).

2.5.2 Precision

The percentage of accurate positive predictions is known as precision. When examining breast cancer diagnosis, precision answers the question: "What proportion of the patients predicted to have cancer actually have it?".

$$Precision = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Positives\ (FP)}$$

High precision is important when the cost of a false positive (predicting cancer when there is not cancer) is high.

2.5.3 Sensitivity (Recall)

The measurement of how successfully a model recognizes positive cases is called sensitivity. Sensitivity is also known as true positive rate or recall. It provides the following response: "Of all the patients who actually have cancer, how many did the model correctly identify?".

$$Sensitivity = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Negatives\ (FN)}$$

In healthcare, high sensitivity is critical because missing a diagnosis (false negatives) can be life-threatening.

2.5.4 Specificity

The percentage of true negatives (patients without cancer) that were accurately identified is known as specificity. Specificity is also known as true negative rate. It offers the following response: "Of all the patients who do not have cancer, what proportion did the model correctly classify as cancer-free?".

$$Specificity = \frac{True\ Negatives\ (TN)}{True\ Negatives\ (TN) + False\ Positives\ (FP)}$$

High specificity is essential when false positives (incorrectly diagnosing cancer) can cause unnecessary stress and treatments.

2.5.5 Receiver Operating Characteristic (ROC) Curve

The trade-off between sensitivity (true positive rate) and 1-specificity (false positive rate) is represented graphically by the ROC curve at different threshold values. Each point on the curve indicates a sensitivity/specificity pair assigned to a specific judgment threshold. Furthermore, the model performs better the closer the ROC curve is near the upper left corner.

The measurement of the model's performance across all classification levels is done with the Area Under the ROC Curve (AUC). It shows the False Positive Rate (1 - Specificity) vs the True Positive Rate (Sensitivity). The range of AUC is from 0 to 1, where 0.5 represents random guessing and 1 is a perfect classifier. An increased AUC means better model performance.

2.5.6 F1 Measure

The harmonic mean of recall and precision is called F1 Score, which balances the two measurements. When attempting to find a compromise between recall and precision, it can be helpful, particularly when the classes are uneven.

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

When both false positives and false negatives need to be minimized, the F1 score can be especially helpful.

2.6 Development tools

This thesis incorporates several tools for the system's development. Python was used for data preprocessing, and ProbLog was applied for the description and construction of the training model. React was utilized to build the user interface, while Flask, a Python web framework, was employed to deploy the system as a web application. The following subsection provides detailed descriptions of these tools.

2.6.1 Python

Python is a versatile, high-level programming language that is easy to learn and read. It is an interpreted language that accommodates various programming styles, such as procedural, object-oriented, and functional programming, which contributes to its popularity across diverse application areas. Python's extensive standard library, along with its numerous third-party packages, provides robust tools for works such as machine learning, web development, data analysis and more. Its user-friendly syntax and dynamic typing contribute to rapid development and prototyping, making Python particularly appealing for researchers and developers in the field of machine learning and data science [23].

2.6.2 ProbLog

Probabilistic Logic Programming (ProbLog) is a probabilistic variant of logic programming that blends logic programming and probabilistic reasoning [2]. It provides a logical framework for the representation of uncertain and probabilistic knowledge, making it possible to model complicated relationships and uncertainty declaratively. ProbLog augments logical rules with probabilities, which apply weights to several alternative interpretations of the rules. These probabilities describe the uncertainty surrounding the truth values of the predicates in the rules and are used to calculate the likelihood of various outcomes.

ProbLog's syntax is similar to that of classic logic programming languages like Prolog, but with probability annotations added to predicate formulations instead [2]. A typical ProbLog program consists of a set of rules and facts, with each rule assigned a probability.

For instance, consider the following ProbLog program:

```
1  0.5::coin(tails).
2  0.5::coin(heads).
3
4  0.6::biased_coin(tails).
5  0.4::biased_coin(heads).
6
7  0.8::biased_flip(tails).
8
9  flip(tails):- coin(tails).
10 flip(tails):- biased_flip(tails).
11 flip(heads):- coin(heads).
12 flip(heads):- biased_flip(heads).
13
14 query(flip(tails)).
```

Figure 2: CoinFlip ProbLog programme

In this example, the program models two coins, a conventional one with equal odds of landing on heads or tails, as well as a biased one with a larger chance of landing on tails. The `biased_flip/1` predicate describes the outcome of flipping a biased coin, whereas the `flip/1` predicate reflects the outcome of flipping any coin. The predicate `query/1` is a goal which requests the likelihood of flipping tails.

ProbLog has applications in a variety of fields, including probabilistic reasoning, probabilistic databases, natural language processing, bioinformatics, and decision support systems. It offers a versatile and expressive framework for modeling uncertain and probabilistic knowledge, allowing for the integration of logical and probabilistic reasoning in a single framework.

Despite its expressive strength, ProbLog faces scalability and efficiency issues, particularly when working with large-scale probabilistic models and sophisticated inference tasks. To address these problems and realize ProbLog's full promise in practical applications, researchers continue to work on developing efficient inference algorithms and scalable probabilistic programming frameworks.

2.6.3 React

React [24] is a popular JavaScript framework that is commonly utilized for creating user interfaces, especially for single-page applications that require fast, efficient, and responsive user experiences. Originally developed and maintained by Facebook, along with a vast community of individual developers and companies, React has become one of the most popular tools of modern web development.

React offers several key features that make it a powerful tool for building user interfaces. Its component-based architecture promotes breaking down the UI into self-contained, reusable components, making development, testing, and maintenance easier. React's declarative syntax allows developers to specify what the UI should look like, and it updates the view automatically when data changes. By using a virtual Document Object Model (DOM), React optimizes performance by first updating the virtual DOM and then efficiently updating the actual DOM based on the differences, minimizing costly DOM manipulations. Additionally, React enforces one-way data binding, ensuring that data flow and application logic are more predictable and easier to understand.

React is chosen for its exceptional performance, flexibility, and strong ecosystem. Its virtual DOM and efficient update mechanism ensure high performance even in complex applications with frequent UI updates. React can be used to build a wide variety of applications, from simple static websites to intricate web applications, and it integrates well with other libraries and frameworks, offering great flexibility in project structure. Additionally, React boasts a vast ecosystem of tools, libraries, and community support, providing developers with extensive resources for solving problems and enhancing their applications. Its component-based architecture also makes it easier to scale applications, allowing for the reuse of components and maintaining a manageable codebase as the application grows.

2.6.4 Flask

Flask is a lightweight and flexible web framework for Python. It is designed to make it easy to start with simple web applications and scale them up to more complex ones as needed. Flask is a micro-framework, meaning it provides the essential tools to build web applications but does not come with many pre-built components. This allows developers to choose their own components and libraries, making it highly modular and extensible. You can add features and functionalities using various Flask extensions, which provide integration for database handling, form validation, authentication, and more [25].

One of the main advantages of Flask is its simplicity and flexibility. Flask is really handy for web development beginners, because of its simple and straightforward syntax, while its flexibility also allows experienced developers to build more complex applications. Flask also provides tools for creating RESTful APIs. The routing system in Flask allows for clean URL mapping and handling of HTTP methods (GET, POST, PUT, DELETE).

Flask comes with a built-in development server and debugger. The debugger provides detailed error messages and an interactive traceback, which are useful during development. Flask uses Jinja2 as its templating engine, allowing you to create dynamic HTML pages with template inheritance and other powerful features.

3 Related Work

The collected and studied related work uses the Breast Cancer Wisconsin diagnostic data set and is focused on using different machine learning algorithms such as Decision Tree (C4.5), Naive Bayes (NB), k Nearest Neighbors (k-NN) and Support Vector Machine (SVM). It is also focused on assessing the efficiency and effectiveness of each algorithm based on its accuracy, precision, sensitivity and specificity.

The work of Asri et al. [26] explores the performance of different machine learning algorithms (Decision Tree, k-NN, SVM, Naive Bayes) on the Wisconsin Breast Cancer dataset, aiming to gauge their accuracy, precision, sensitivity, and specificity. Support Vector Machine (SVM) stands out as the most efficient and effective, achieving the highest accuracy (97.13%) and lowest error rate. Conducted using the WEKA data mining tool, the study underscores SVM's superiority in classifying breast cancer data, surpassing other algorithms in accuracy, sensitivity, specificity, and precision, achieving an accuracy rate of 97.28%.

Adebiyi et al. [27] aimed to improve the precision for breast cancer diagnosis in machine learning algorithms by utilizing techniques like Linear Discriminant Analysis (LDA) for feature extraction. Using the Wisconsin Breast Cancer Dataset, the study applies SVM and Random Forest (RF) algorithms, achieving 96.4% accuracy with SVM-LDA and 95.6% with RF-LDA. By reducing data complexity and extracting essential features, LDA contributes to better prediction accuracy, potentially leading to life-saving advancements in breast cancer detection. The work highlights the importance of further exploring feature extraction methods to enhance predictive models for breast cancer diagnosis.

Naji et al. [28] have also found SVM to be the most effective algorithm, achieving 97.2% accuracy, 97.5% precision, and 96.6% AUC, when assessing five machine learning algorithms (K-NN, Logistic Regression, Decision Tree, SVM, Random Forest) using the Wisconsin Breast Cancer Diagnostic dataset (WBCD) to determine the most accurate and dependable model.

Tomar et al. [29] have created a Hybrid Feature Selection (HFS) model for diagnosing Breast Cancer, Hepatitis, and Diabetes, aiming to tackle class imbalance and feature selection problems. It merges Weighted Least Squares Twin Support Vector Machine (WLSTSVM) classification, sequential forward selection (SFS) search strategy, and correlation feature selection (CFS) and provides an improved accuracy of 98.55%.

Additionally, Abdar et al. [30] have developed classifiers combining different classification algorithms. For the conduction of experiments with the use of K-fold Cross Validation, the WDBC dataset was used. The proposed models were contrasted to individual classifiers (Naive Bayes and BayesNet) in terms of various metrics including recall, precision, accuracy, ROC, F1 measure, and computational times. The findings show that the layered ensemble models with two layers outperform most previous works and individual classifiers, with SV-Naive Bayes-3-MetaClassifier achieving 98.07% accuracy in less time.

Sharma et al. [31] and Kiliç et al. [32] also compared popular machine learning algorithms for breast cancer prediction (Naive Bayes, Random Forest, kNN) using the Wisconsin Diagnosis Breast Cancer dataset. Results showed high accuracy (>94%) for all algorithms, with kNN performing best in detecting breast cancer based on accuracy, precision, and F1 score. The second group found the kNN algorithm performing with 99.3% accuracy and the SVM and Logic Regression algorithms as the second most successful models.

Using machine learning methods, the Wisconsin breast cancer dataset was visually analyzed and tumor kinds (malignant or benign) were categorized by Ak M. [33]. Three datasets—one with all features, one with strongly correlated features, and one with low correlation—were produced after data visualization. Numerous machine learning techniques were used for classification, with logistic regression yielding the best accuracy (98.1%). The work highlights the need for pre-processing the dataset and handling missing values before applying machine learning algorithms.

In conclusion, the related work has presented the exploration of a range of techniques to enhance predictive accuracy and efficiency ranging from hybrid feature selection models to two-layer nested ensemble classifiers. Comparisons of algorithms such as kNN, Naïve Bayes, Support Vector Machine and Random Forest Bayes have demonstrated promising results, with some models achieving accuracies exceeding 98%. Additionally, visualizing datasets and pre-processing them are highlighted as crucial steps to improve algorithm performance.

4 Methodology

4.1 The Dataset

The dataset used for the creation of the diagnostic model in this system is the Wisconsin Breast Cancer Dataset [1]. This dataset is widely recognized and accepted within the research community, providing a robust foundation for the results presented in this thesis. The elements of this dataset are computed from a computerized picture of a breast mass's fine needle aspiration (FNA). These elements describe the attributes of the cell nuclei that appear in the image. Each instance in the dataset represents a patient's breast cancer diagnosis, with features derived from FNA images. The dataset consists of a total of 569 instances, with 32 attributes for each instance covering various measures such as *radius*, *texture*, *perimeter*, *area*, *smoothness*, *compactness*, *concavity*, *concave points*, *symmetry*, and *fractal dimension*. Lastly, the dataset is divided into two classes: *benign* and *malignant* cases.

Given the large number of attributes, it was necessary to select a subset of the most relevant features for creating the model. After careful and thorough research, it was concluded that five attributes were the most vital for the breast cancer diagnosis model. These attributes are *smoothness_mean* (mean of local variation in radius lengths), *radius_mean* (mean of distances from center to points on the margins), *concave_points_mean* (the average of the contour's concave sections), *symmetry_mean* (mean of symmetry), and *texture_mean* (mean of gray-scale values).

4.2 The Bayesian Network

In order to create a diagnostic Bayesian model, there is a need for the construction of the Bayesian network of the system. Since there has already been a selection of the most significant characteristics in the dataset, these characteristics are used to form the Bayesian network of the system. Specifically, the attributes *radius_mean*, *smoothness_mean*, *concave_points_mean*, *symmetry_mean*, and *texture_mean* are directly related to the nature of the examined cell nuclei, determining their benign or malignant status. Consequently, the Bayesian network consists of two layers: the first layer includes these selected characteristics, and the second layer represents the Breast Cancer predicate. Moreover, the structure of the Bayesian network is illustrated in Figure 3. The direction of the arrows in the diagram denotes the dependencies between the characteristics and the Breast Cancer predicate. These arrows illustrate the causal relationships, indicating how each characteristic influences the likelihood of a benign or malignant diagnosis.

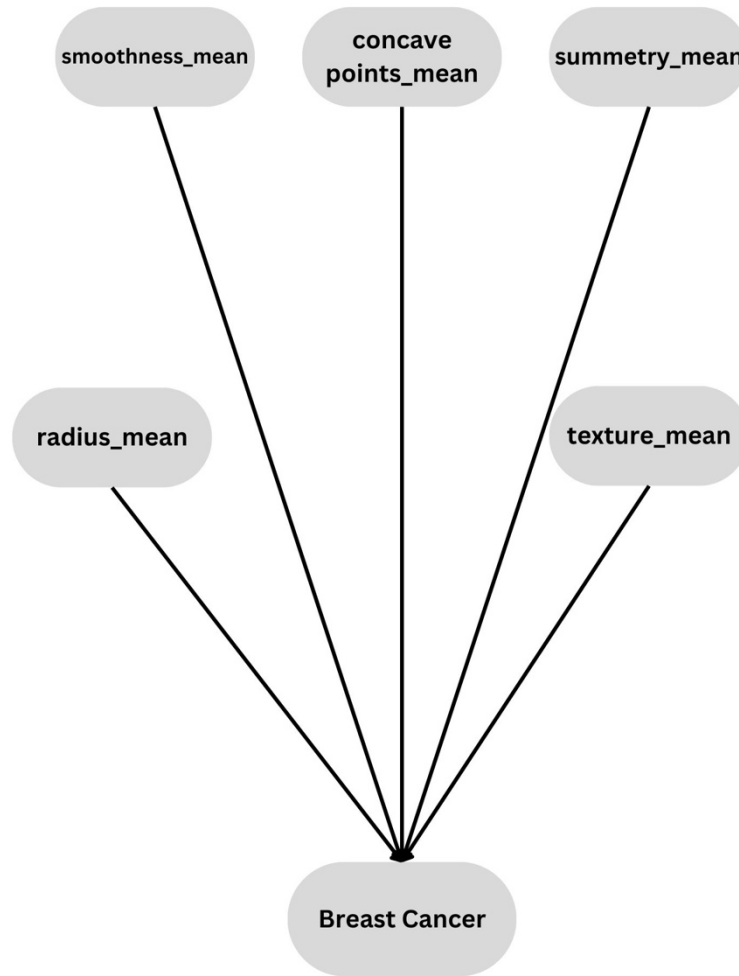


Figure 3: Bayesian Network Diagram

This structure encapsulates the probabilistic relationships between the characteristics of cell nuclei and breast cancer diagnosis, allowing for efficient computation of posterior probabilities. The simplicity of having only two layers enhances the interpretability and applicability of the model in a clinical setting. The Bayesian network serves as the core of the diagnostic system, leveraging these probabilistic dependencies to provide accurate and interpretable diagnoses. This network is fundamental to the subsequent phases of model construction, training, and testing, ensuring that the system can effectively learn from the data and make reliable predictions.

4.3 The Architecture of the Machine Learning System

Following the construction of the Bayesian Network that outlines the structure of the diagnostic model, the first step towards the development of the system is the design of its high-level architecture. This step allows for the definition of the most vital parts of the system and the process of communication between them, highlighting which components will interact with each other. Additionally, this approach helps in selecting the best methods of communication between the components, depending on the needs of the application.

The system consists of two main parts: 1) the machine learning phase and 2) the application phase. For this system, a client-server application approach was deemed ideal. This architecture provides the user with a friendly interface where they can input their data and receive the diagnosis result in an easy-to-understand manner.

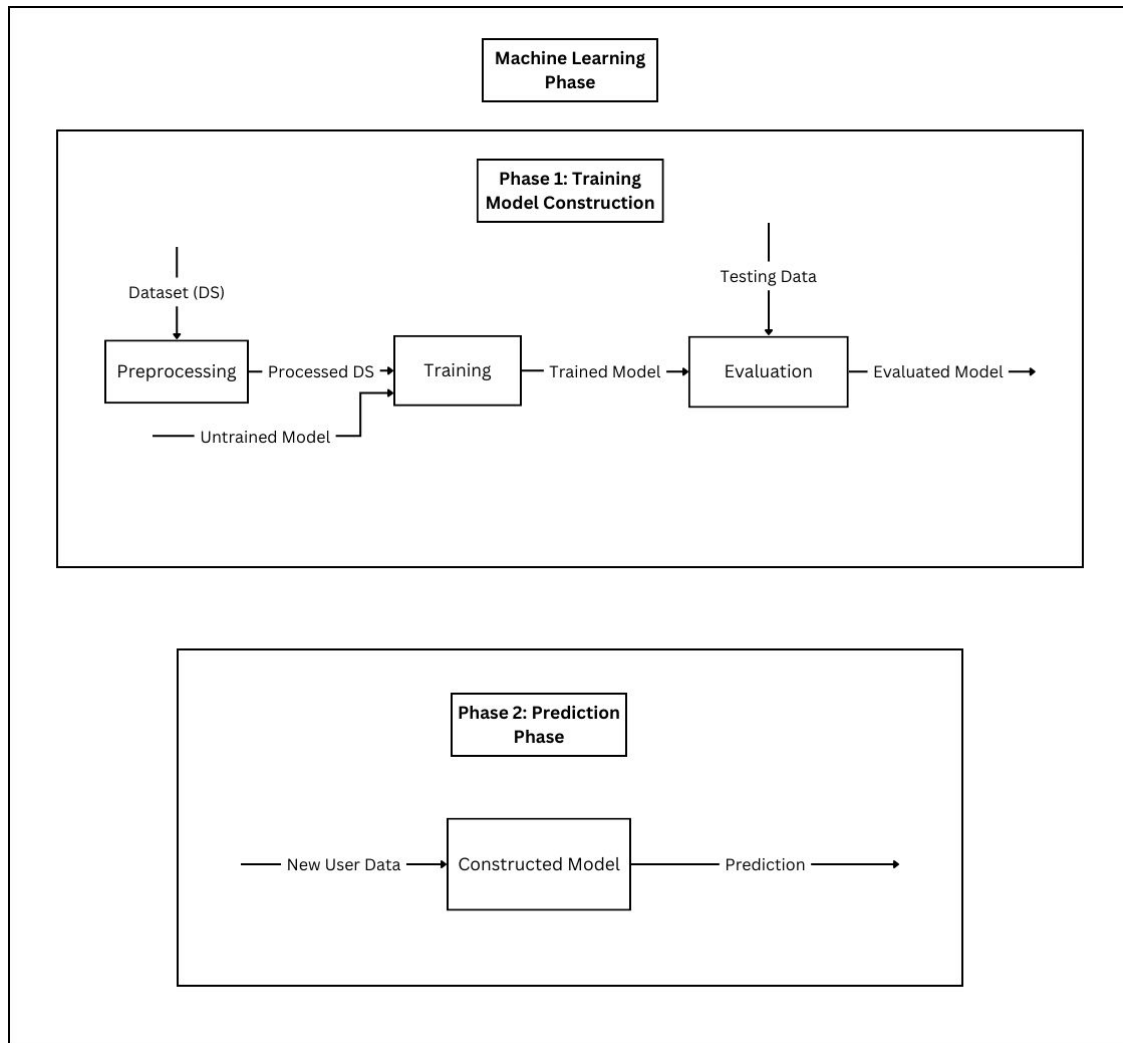


Figure 4: Machine Learning Phase - System Architecture Diagram

The “Machine Learning Phase” is divided into two sub-phases. **Phase 1** is the Training Model construction. It starts with acquiring the dataset and preprocessing it. Preprocessing can include various methods such as removing empty entries and outliers. The preprocessed dataset is then used in the training process along with the constructed untrained model. The training process produces the trained model, which is subsequently evaluated using testing data. The final product of the “training model construction” phase is the evaluated model. **Phase 2** is the Prediction phase, where the user provides new data, and the system uses the constructed model to provide predictions.

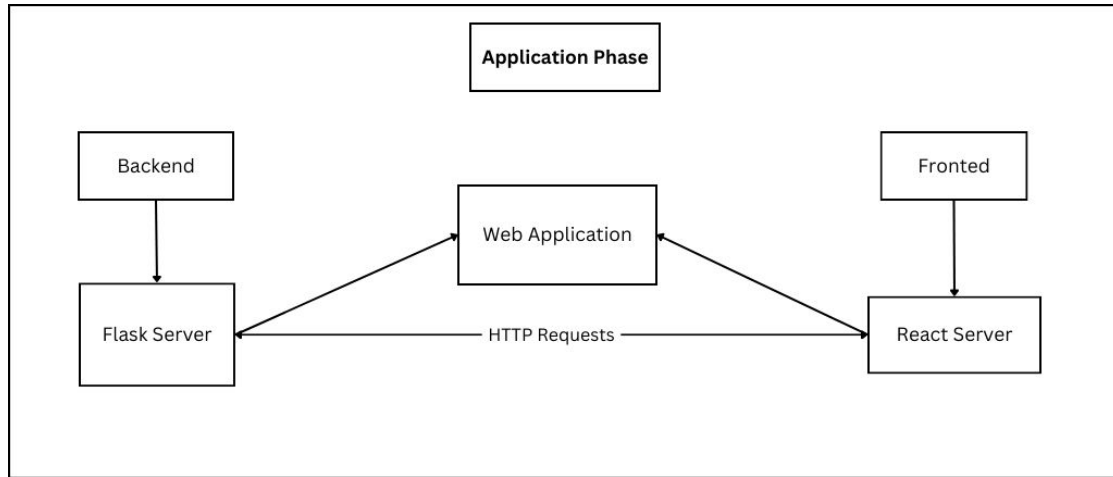


Figure 5: Application Phase - System Architecture Diagram

In the “Application Phase”, the system follows a basic web application format using the Flask and React libraries. The backend part of the application is created using a Flask server, while the frontend part is created with a React server. The backend also includes the “Machine Learning Phase”, providing a space for that part of the system to function. Finally, the two servers communicate with each other through HTTP requests, providing both the system and the user with the necessary information and forming the complete web application.

4.4 Preprocessing

In order to improve the predictive power of our system, several preprocessing methods were used to evaluate the weight of each feature of the input data in the cause of cancer. Each method tried to find the optimal set of features which maximize the performance and accuracy of the model. Overall, the preprocessing approaches aimed to ensure that the selected features from the dataset were appropriately selected for the machine learning analysis task, leading to a more robust and accurate diagnostic model.

The goal of preprocessing is to prepare the raw data for machine learning analysis and model creation. This section presents various preprocessing methods such as feature selection, dropping duplicates, etc. Effective preprocessing guarantees the dataset's *consistency*, *cleanliness*, and *suitability for training* machine learning algorithms, thereby improving the reliability and accuracy of the prediction model. Our goal for this system was to achieve high rates of diagnosis accuracy. Therefore, 5 different preprocessing approaches were tested. The first preprocessing approach involved the following. 1) Dropping duplicates and choosing features based on their importance in affecting the diagnosis results. Although the dataset did not contain any duplicate entries, removing potential duplicates is considered good practice. We added this feature in order to be applied in other datasets that would be used for training other models. 2) The letters ‘B’ standing for benign and ‘M’ standing for malignant in the diagnosis column of the dataset were replaced with the numbers ‘0’ and ‘1’ respectively to facilitate later processing steps. 3) The column name “concave points_mean” was renamed to “concave_points_mean” to avoid issues caused by the spacebar. The selected features of this preprocessing approach are the following: *smoothness_mean*, *symmetry_mean*, *concave points mean*, *radius mean*, *symmetry mean*, and *texture mean*. A sample of 10 instances derived from the preprocessing of this approach are shown in Figure 6.

diagnosis	radius_mean	texture_mean	smoothness_mean	concave_points_mean	symmetry_mean
1	17.99	10.38	0.1184	0.1471	0.2419
1	20.57	17.77	0.08474	0.07017	0.1812
1	19.69	21.25	0.1096	0.1279	0.2069
1	11.42	20.38	0.1425	0.1052	0.2597
1	20.29	14.34	0.1003	0.1043	0.1809
1	12.45	15.7	0.1278	0.08089	0.2087
1	18.25	19.98	0.09463	0.074	0.1794
1	13.71	20.83	0.1189	0.05985	0.2196
1	13.0	21.82	0.1273	0.09353	0.235
1	12.46	24.04	0.1186	0.08543	0.203

Figure 6: Sample of the first 10 instances of the cleaned dataset using the first approach.

The second preprocessing approach involved two tests, as follows. 1) All the "mean" values in the dataset were retained, yielding a set of ten features for analysis. This provided a broad foundation for feature selection. However, the accuracy results of this method were lower than the results of the initial approach, therefore it was not further examined. 2) Keeping the features selected in the initial approach, which were also chosen for the Bayesian model, and adding the *compactness_mean* feature to enhance predictive power with an additional relevant characteristic. After testing the model didn't appear to have any improvement in its results, therefore the *compactness_mean* feature was removed. The second preprocessing approach is ignored and we keep the first one up to this stage of experimentation.

The third preprocessing approach involved splitting the dataset into groups of 100 records (e.g., 2-101, 102-202, etc), so each group was further divided in a training set of 90 records and a test set of 10 records. The idea was to identify records which did not contribute to the diagnosis task. However, this method revealed an accuracy of only 36% in the second group, therefore we decided to stop this preprocessing method.

Lastly, a fourth approach was developed for handling the outliers using the Interquartile Range (IQR) method. This statistical approach aimed in identifying and managing outliers, ensuring the dataset's integrity, however its results also proved to be of lower performance than the results of the initial approach.

4.5 Model Construction

Model construction in Bayesian machine learning involves creating a probabilistic framework that allows for resilient and interpretable predictions. Unlike traditional machine learning models, Bayesian models incorporate prior knowledge along with observed data to update beliefs and make predictions. This approach is particularly advantageous in medical diagnosis, where prior probabilities are able to greatly enhance the reliability and accuracy of the model. In this section, the steps taken to construct the Bayesian model for breast cancer diagnosis will be presented in details, including converting numerical data to categorical, data binning, model formulation using ProbLog, and the generation of training and testing datasets. Note that, *data binning*, also called *data bucketing*, is a feature engineering technique that groups different

numerical intervals into bins or buckets. So, data binning is used to map numerical data into categorical data.

The untrained model of our system is constructed using Python. After the data preprocessing, the cleaned .csv file is loaded into a dataframe of Python, and a variable named ‘program’ is created in order to store the untrained model which is a ProbLog program. In order to create the untrained model initial facts are added to the Python variable ‘program’. ProbLog requires that numerical data to be converted to categorical. In order to achieve that, the columns of the csv file are then parsed, and their values are binned and labelled into categories. For example, the diagnosis column takes values in the interval from 0 to 1, i.e. [0-1]. So, any value between 0 and 0.5, i.e. [0-0.5), is labelled as 'n' (no), and any value between 0.5 and 1, i.e. [0.5-1] is labelled as 'y' (yes). In order to select the best categorical set of data values based on the prediction performance of our system, we tested several mappings of real values to different sets of categorical values. We experimented on the following mappings of real values of the data set to corresponding categorical values.

As a first approach, the data values space of each feature was split into three subintervals and then each one has been assigned a value labelled as '**l**' for *low*, '**m**' for *medium*, and '**h**' for *high*. However, the subintervals of each feature were based on its histogram (Figure 7), and the numbers at the edges were rounded to capture certain values. The idea behind this was the following. Each histogram consists of ten bars of different sizes. The size of each bar is based on the number of instances that apply to it. These ten bars were divided into three intervals as follows. The first three bars formed the first interval, the next three bars formed the second interval and the last four bars formed the third interval, because very few instances applied to theses bars.

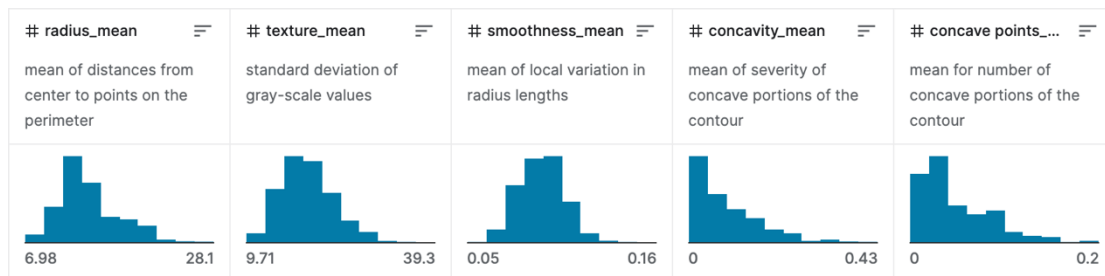


Figure 7: Histogram showing the ranges of the values inside the Dataset.

Following data binning, the partially constructed untrained model was enriched with ProbLog clauses which have all possible combinations of the selected predicates, creating the complete untrained model (Figure 9). The cleaned csv file was then divided into training and testing data sets in ratios 90% and 10% respectively where the records for each dataset were randomly selected. Using the training dataset, our Python code created a string variable named ‘evidence’, where statements regarding the evidence were stored. These statements are created from the training data. For example:

The record of the data set

diagnosis	radius_mean	texture_mean	smoothness_mean	concave_points_mean	symmetry_mean
y	h	l	m	h	h

Figure 8: Dataset record

is mapped into the following set of statements of evidence.

```
evidence(radius_mean(y,h), true).  
evidence(texture_mean(y,l), true).  
evidence(smoothness_mean(y,m), true).  
evidence(concave_points_mean(y,h), true).  
evidence(symmetry_mean(y,h), true).
```

Finally, both the untrained model and the evidence were stored into ProbLog's files named 'untrained_model.pl' and 'training_evidence.pl' respectively.

In an attempt to improve the constructed model, a second approach was followed in which the levels used to describe the intervals were increased from three (3) (low, medium, high) to five (5) (*very low* (**vl**), *low* (**l**), *medium* (**m**), *high* (**h**), and *very high* (**vh**)). Moreover, there was a re-division of the intervals, based on the histogram (Figure 7) to determine the boundaries of each interval. Each interval included approximately 2-3 bars of the histogram. However this approach didn't show any promising improvement on the model's performance.

Finally, a third model construction approach was followed, where an algorithm was developed for automatic data binning interval creation based on the number of desired intervals. The creation of intervals involved collecting the min and max values of each characteristic and then dividing the range by the number of desired subintervals. Necessary adjustments were made to determine the boundaries of the intervals for each characteristic. Correspondingly, labels for the intervals were created based on the number of subintervals. These labels are created by combining the character "v" and the number of each subinterval, for example "**v0**", "**v1**", "**v2**", etc. This method was tested with interval creation ranging from 2 to 10 subintervals. The one which resulted in the best performance of our system is the splitting on initial intervals into 3 subintervals, so this approach has been incorporated into our system. So, in this case the values of subintervals "**v0**", "**v1**" and "**v2**" correspond to values *low*, *medium*, and *high* respectively as in the case of the first approach.

Example

The untrained model shown in Figure 9 illustrates the selected data binning approach, that is the third approach of 3 subintervals.

```

1 res(n). res(y). level(v0). level(v1). level(v2).
2 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
3 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
4 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
5 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
6 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
7 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
8 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
9 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
10 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
11 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
12 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
13 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
14 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
15 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
16 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
17 diagnosis(A,B, C, D, E, F):- \+radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
18 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
19 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
20 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
21 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
22 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
23 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
24 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
25 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
26 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
27 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
28 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
29 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
30 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
31 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
32 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
33 diagnosis(A,B, C, D, E, F):- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
34 t(0.5,A,B):-radius_mean(A,B), level(B).
35 t(0.5,A,B):-texture_mean(A,B):- level(B).
36 t(0.5,A,B):-smoothness_mean(A,B):- level(B).
37 t(0.5,A,B):-concave_points_mean(A,B):- level(B).
38 t(0.5,A,B):-symmetry_mean(A,B):- level(B).

```

Figure 9: The untrained model that has been incorporated to the system.

The untrained model consists of predicates describing the results of the diagnosis based on the selected features and the “levels” representing the number of intervals used in the binning process. Moreover, it consists of all the possible combinations between the diagnosis of the system and the attribute predicates of the system. Lastly, there is the *t* predicate of ProbLog that is used for parameter learning, using the EM algorithm.

4.6 Training

The training process is crucial for generating the trained Bayesian model for breast cancer diagnosis. This involves parameter learning using the Expectation-Maximization (EM) algorithm, which iteratively refines the parameters of the model to optimize the probability of the data that was observed. Initially, each probability in the model is set to a default value as specified in the probabilistic facts. For this model, the initial value is set to **0.5**, represented in the code as *t(0.5, A, B)*. This initial value serves as the starting point for the EM algorithm. The EM algorithm iteratively adjusts these probabilities to better fit the data, converging on the most likely values that explain the training data. In order to implement the training, the **Learning from Interpretations (LFI)** library of the ProbLog framework is used which is integrated into Python. The following command is executed in the terminal to create the trained model:

```
ProbLog lfi untrained_model.pl training_evidence.pl -O trained_model.pl
```

This command takes the untrained model (‘untrained_model.pl’) and the training evidence (‘training_evidence.pl’) as inputs and outputs the trained model (‘trained_model.pl’) into the source folder. This trained model now contains the learned parameters that is useful for making predictions on new data. The training process was consistent across the different approaches of improving the system.

```

1 res(n).
2 res(y).
3 level(v0).
4 level(v1).
5 level(v2).
6 diagnosis(A,B,C,D,E,F) :- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
7 diagnosis(A,B,C,D,E,F) :- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
8 diagnosis(A,B,C,D,E,F) :- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
9 diagnosis(A,B,C,D,E,F) :- \+radius_mean(A,B), \+texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
10 diagnosis(A,B,C,D,E,F) :- \+radius_mean(A,B), \+texture_mean(A,C), smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).

```

Figure 10: Sample of Trained Model. (1/2)

...

```

31 diagnosis(A,B,C,D,E,F) :- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
32 diagnosis(A,B,C,D,E,F) :- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
33 diagnosis(A,B,C,D,E,F) :- radius_mean(A,B), texture_mean(A,C), \+smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
34 diagnosis(A,B,C,D,E,F) :- radius_mean(A,B), texture_mean(A,C), smoothness_mean(A,D), \+concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
35 diagnosis(A,B,C,D,E,F) :- radius_mean(A,B), texture_mean(A,C), smoothness_mean(A,D), \+concave_points_mean(A,E), symmetry_mean(A,F), res(A).
36 diagnosis(A,B,C,D,E,F) :- radius_mean(A,B), texture_mean(A,C), smoothness_mean(A,D), concave_points_mean(A,E), \+symmetry_mean(A,F), res(A).
37 diagnosis(A,B,C,D,E,F) :- radius_mean(A,B), texture_mean(A,C), smoothness_mean(A,D), concave_points_mean(A,E), symmetry_mean(A,F), res(A).
38 0.3::radius_mean(n,v2) :- level(v2).
39 0.7::radius_mean(y,v2) :- level(v2).
40 0.142276422764228::radius_mean(y,v1) :- level(v1).
41 0.857723577235772::radius_mean(n,v1) :- level(v1).
42 0.0::radius_mean(y,v0) :- level(v0).
43 1.0::radius_mean(n,v0) :- level(v0).
44 0.121212121212121::texture_mean(y,v0) :- level(v0).
45 0.878787878787879::texture_mean(n,v0) :- level(v0).
46 0.527881040892193::texture_mean(n,v1) :- level(v1).
47 0.472118959107807::texture_mean(y,v1) :- level(v1).
48 0.341463414634146::texture_mean(n,v2) :- level(v2).
49 0.658536585365854::texture_mean(y,v2) :- level(v2).

```

Figure 11: Sample of Trained Model. (2/2)

By using the EM algorithm and the LFI library, the model parameters are iteratively refined to improve the reliability and accuracy of the breast cancer diagnosis predictions. This systematic approach to training ensures that the model is well-calibrated and capable of making robust predictions based on the input data. Moreover, the model is interpretable due to Bayesian modelling.

Example

A sample of training evidence is following. This training evidence is part of the data set that we used. This example illustrates the representation of the data that are given to lfi function. Note that, the values **v0**, **v1** and **v2** correspond to *low*, *high* and *medium* data values as specified by the corresponding sub-intervals.

```

1 evidence(radius_mean(n,v0), true).
2 evidence(texture_mean(n,v1), true).
3 evidence(smoothness_mean(n,v2), true).
4 evidence(concave_points_mean(n,v0), true).
5 evidence(symmetry_mean(n,v1), true).
6 -----
7 evidence(radius_mean(y,v1), true).
8 evidence(texture_mean(y,v1), true).
9 evidence(smoothness_mean(y,v1), true).
10 evidence(concave_points_mean(y,v2), true).
11 evidence(symmetry_mean(y,v1), true).
12 -----

```

Figure 12: Sample of Training Evidence.

The evidence includes the association of each attribute and its level, to the diagnosis result (yes or no).

4.7 Testing

The evaluation of the machine learning model involves several steps to ensure its accuracy and effectiveness in diagnosing breast cancer. The initial step in this process is to evaluate the trained model against the testing data. This comparison involves querying the trained model with the testing dataset and comparing the outputs with the expected results. This step is critical for understanding how well the model generalizes to new, unseen data. In order to carry out the evaluation, the built-in function ‘evaluate’ of ProbLog Framework is utilized, which performs a detailed analysis of the model's performance. The results of this evaluation are saved in a file named ‘testing_results.txt’ within the dataset folder. This file contains the key performance metrics that help in understanding the model's accuracy and reliability.

The evaluation function is called as follows

```
results = get_evaluable().create_from(p).evaluate()
```

Where the parameter of the call is p that contains the trained model in string form combined with the queries necessary for testing.

Following the primary evaluation, to offer a thorough evaluation of the performance of the model, several metrics are computed. These metrics include *accuracy*, which is the proportion of cases that were accurately predicted to all instances; *recall*, which is the proportion of true positive predictions to the sum of true positive and false negative predictions; the *F1-Score*, which provides a balance between precision and recall by taking the harmonic mean of the two and *precision*, which is the proportion of true positive predictions to the sum of true positive and false positive predictions. In addition to these metrics, the model's performance is further evaluated using the curve analysis of the *Receiver Operating Characteristic (ROC)*. More specifically, the curve of ROC illustrates the difference between the false positive rate (FPR) and true positive rate (TPR) for different threshold values. Additionally, the *Area Under the Curve (AUC)* is also calculated, offering a comprehensive assessment of the model's performance across all categorization levels. An increased AUC value hints enhanced overall performance.

To visualize the ROC curve, the Python library ‘matplotlib.pyplot’ is employed. This visualization helps in being aware of how the true positive rate and false positive rate are traded off and aids in selecting the optimal threshold for classification. A confusion matrix is computed and displayed, providing a more detailed interpretation of the model's predictions. The confusion matrix provides a tabular summary of the model's prediction results and highlights the number of true positive, true negative, false positive, and false negative predictions. This matrix is essential for identifying specific areas where the model may be underperforming and requires further improvement.

4.8 Application Phase

The Application Phase of the system involves the integration of the trained machine learning model into a practical and user-friendly application. This phase ensures that end-users can interact with the system efficiently, inputting data and receiving diagnostic results seamlessly. The application is divided into two main components: the front-end, which handles the user

interface and experience, and the back-end, which manages the machine learning process, communication with the trained model, and serving results.

4.8.1 Front-End

The application's frontend has been built using the React framework. It includes three primary web pages: a) the Home page, b) the Knowledge Engineer Page, and c) the ordinary User Page. On the Home page (Figure 13), users are greeted with two buttons that allow navigation to either the Knowledge Engineer Page or the ordinary User Page, making the interface straightforward and easy to use.

Breast Cancer Prediction App

Select a Profile:

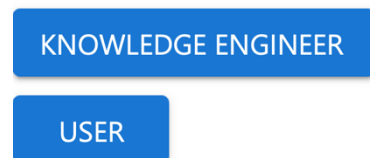


Figure 13: Application's Home Page

The Knowledge Engineer page (Figure 14) is equipped with the following four functional buttons.

1. Prepare Dataset: This button runs a Python script ('preparing_dataset.py') to preprocess and prepare the dataset for further analysis.
2. Create Model: By clicking this, another script ('creating_model.py') is executed to generate an untrained model and training evidence.
3. Evaluate Performance: This triggers the script ('testing_model.py') to evaluate the model's performance, producing pop-up window plots and a 'testing_results.txt' file for detailed analysis.
4. Home: This redirects the user back to the Homepage, ensuring seamless navigation between different sections of the application.

Knowledge Engineer Menu

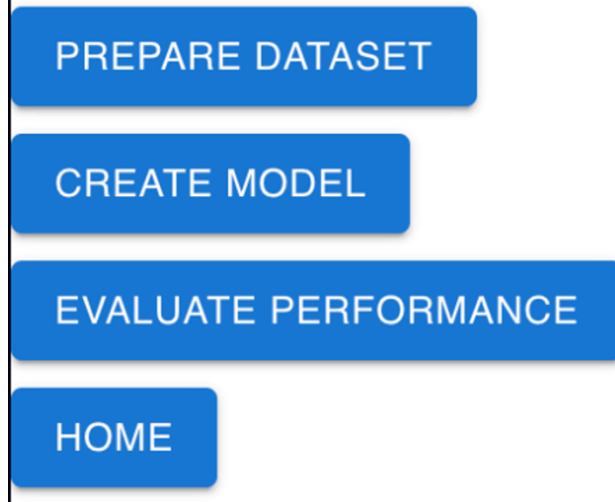


Figure 14: Knowledge Engineer Menu

The User page (Figure 15) features a form with five input fields: a) Radius Mean, b) Texture Mean, c) Smoothness Mean, d) Concave Points Mean, and e) Symmetry Mean. Users enter information regarding the characteristics of the cell nuclei and submit the form. Upon submission, the frontend communicates with the backend via an API call, which processes the input data and returns probabilities indicating the likelihood of having (y) or not having (n) breast cancer. This real-time feedback is crucial for timely and accurate diagnosis.

Please enter your information:

Radius Mean(0.10-29.00)

Texture Mean(0.10-40.00)

Smoothness Mean(0.01-0.20)

Concave Points Mean(0.0-0.30)

Symmetry Mean(0.10-0.35)

Figure 15: User Page

To initiate the frontend server, run "npm start" in a separate terminal window within the frontend directory. Before starting the server, ensure all necessary dependencies are installed using the "npm install" command. If everything is set up correctly, a new browser tab will open,

displaying the Homepage, ready for user interaction. This setup process is designed to be as simple as possible, allowing users to quickly get the application up and running.

Please enter your information:

Radius Mean(0.10-29.00)

Texture Mean(0.10-40.00)

Smoothness Mean(0.01-0.20)

Concave Points Mean(0.0-0.30)

Symmetry Mean(0.10-0.35)

Diagnostics

y: 0.03, n: 1.00

Figure 16: System results example.

Figure 16 is an example of the result that our system returns when provided with real data. In this particular case, the prediction says that the potential patient with probability 0.03 has breast cancer, and with probability 1.00 has not breast cancer.

4.8.2 Back-End

The backend of the application is built using Flask, a lightweight Python web framework ideal for creating APIs. The core of the backend resides in a single Python file named "backend.py," which handles all the essential operations and logic of the application. The backend server starts by running the command 'python3 backend.py' in a new terminal window within the backend directory. This action initializes the Flask server, which then listens for incoming HTTP requests from the frontend. It is crucial to run both the frontend and backend servers simultaneously in separate terminal windows to ensure the application functions correctly.

The backend.py file is responsible for handling all HTTP requests between the backend and the frontend of the application. When a user inputs data on the client page and submits it, the frontend sends this data via an API call to the backend. The backend then processes this data, utilizing the trained model to predict the likelihood for the presence of breast cancer, and sends the results back to the frontend to be displayed to the user.

5 Results and Discussion

This chapter presents the results of the Bayesian network model for breast cancer diagnosis, analyses these results, and discusses their significance. The model's performance metrics are evaluated, and a comparative analysis with existing models is provided. The performance of the Bayesian network model is evaluated using various metrics, including *precision*, *accuracy*, *F1-score*, *recall* and *the ROC curve*.

Our goal was a high-performance diagnosis system, therefore we tried several approaches regarding the development of the Bayesian network model, both at the stage of preprocessing of the data set as well as at the stage of the construction of the trained model. The most efficient one has been proved to be the first data preprocessing approach, combined with the third model construction approach, as it also is the most adaptable. This combination included the initial selection of characteristics meaning *smoothness_mean*, *symmetry_mean*, *concave_points_mean*, *radius_mean* and *texture_mean*, with the steps of initial data preprocessing combined with the creation and use of the automatic binning algorithm in the model construction stage. The most ideal number of intervals proved to be 3, after evaluating several possible numbers of interval values, i.e. 2-10. The third approach of model construction performed a precision of 89%, an accuracy of 89%, and a recall of 89%. The F1-score, which balances precision and recall, was also 89%. These metrics are summarized in Table 1.

Table 1: Model Performance

Metric	Value
Accuracy	89%
Precision	89%
Recall	89%
F1-score	89%

The ROC curve, shown in Figure 17, illustrates the ratio of the false positive rate (1-specificity) to the real positive rate (sensitivity) at certain threshold values. The Area Under the Curve (AUC) was found to be 97%, indicating a strong discriminatory ability of the model.

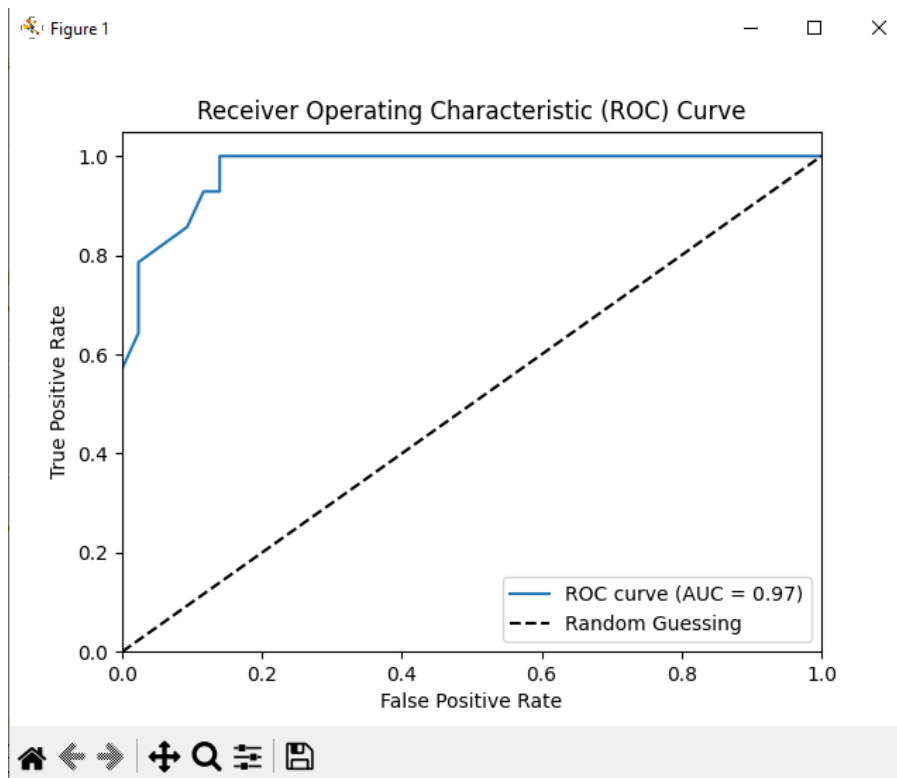


Figure 17: The model's ROC Curve

The confusion matrix in Figure 18 demonstrates how true positives, true negatives, false positives, and false negatives are distributed. The model correctly identified 13 true positives and 38 true negatives, while there were 5 false positives and 1 false negatives.

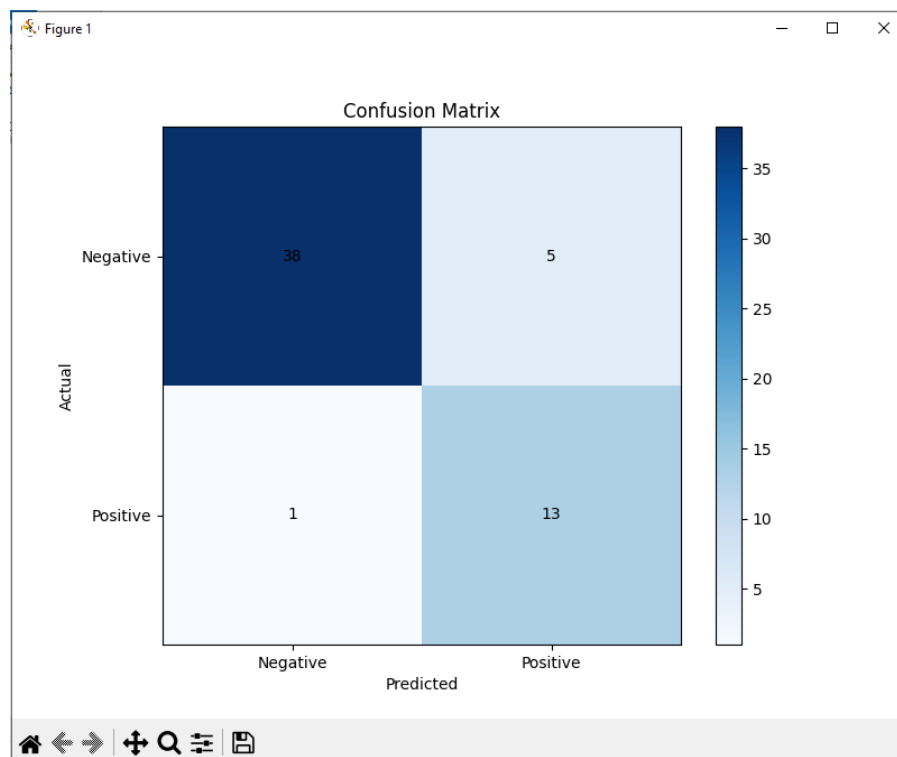


Figure 18: The model's Confusion Matrix

Table 2: The model's Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	13	1
Actual Negative	5	38

When compared to other models, studied in the related work section, such as Support Vector Machine and Random Forest, the Bayesian network model demonstrated competitive performance. While the other models showed higher accuracy, the Bayesian approach provides better interpretability and insight into the relationships between features. The model's performance indicates a robust ability to distinguish between benign and malignant breast cancer cases. The high precision suggests that true positive examples are successfully identified by the model, while the recall indicates that most actual positive cases are correctly identified. The results imply that this model can be a valuable tool for healthcare practitioners, providing reliable diagnostic predictions. This could potentially lead to earlier detection and treatment of breast cancer, improving patient outcomes.

6 Conclusion and Future Work

This thesis presented the development of a diagnosis system for Breast Cancer. The system is based on Probabilistic logical reasoning using Bayesian Networks. These networks are adept at modeling probabilistic relationships and managing uncertainty, making them a promising tool for medical diagnostics. Although other machine learning techniques can yield higher accuracy, this thesis highlights the crucial role of *interpretability* and *transparency* in Bayesian Networks. The diagnostic model was constructed using the Wisconsin Breast Cancer Dataset, a well-regarded dataset in the research field. This dataset is based on digitized images of fine needle aspirates (FNA) from breast masses, providing detailed attributes of cell nuclei. A thorough analysis identified five critical features essential for the diagnostic model: `smoothness_mean`, `symmetry_mean`, `concave_points_mean`, `radius_mean` and `texture_mean`.

The system includes two main phases, namely the *machine learning phase* and the *application phase*. The *machine learning phase* involves training and testing the model with the dataset, including performance evaluation, data preprocessing and model training. The resulting evaluated model is then used for predictions, allowing new data inputs to generate diagnostic results. In the *application phase*, the system uses a web interface which has been built using Flask for the backend and React for the frontend. These components interact via HTTP requests, offering a smooth user experience. The system provides two primary interfaces: one for users (potential patients) to receive diagnostic results and another for the knowledge engineer to manage the machine learning processes. Among several approaches tested, the most effective involved initial data preprocessing and the application of an automatic binning algorithm. This method, which utilized three intervals, achieved **89%** in *accuracy*, *precision*, and *recall*, along with an *F1-score* of **89%**. The ROC curve analysis yielded an Area Under the Curve (AUC) of **97%**, indicating strong model performance in distinguishing between benign and malignant cases.

When compared our model to other models like Support Vector Machines and Random Forests, the Bayesian network model showed *competitive results*, with the added benefit of being more *interpretable*. This interpretability is particularly valuable in medical contexts, where understanding the basis for predictions is essential for healthcare professionals. This system represents a significant tool for the early detection of breast cancer, potentially leading to better patient outcomes through timely diagnosis and treatment. Future enhancements could focus on improving model accuracy, refining the user interface to provide the reasoning behind the diagnostic results to users, and implementing user authentication to differentiate access rights for users and for the knowledge engineer.

7 Bibliography

- [1] “Breast Cancer Wisconsin (Diagnostic) Data Set.” Accessed: Nov. 22, 2023. [Online]. Available: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
- [2] L. De Raedt and A. Kimmig, “Probabilistic (logic) programming concepts,” *Mach Learn*, vol. 100, no. 1, pp. 5–47, Jul. 2015, doi: 10.1007/s10994-015-5494-z.
- [3] J. R. Koza, F. H. Bennett Iii, D. Andre, and M. A. Keane, “Automated Design of Both the Topology and Sizing of Analog Electrical Circuits Using Genetic Programming,” in *Artificial Intelligence in Design '96*, Springer Netherlands, 1996, pp. 151–170. doi: https://doi.org/10.1007/978-94-009-0279-4_9.
- [4] T. M. . Mitchell, *Machine learning*. McGraw-Hill, 1997.
- [5] E. Alpaydin, *Introduction to Machine Learning*, Second Edition. The MIT Press, 2009.
- [6] Z. H. Zhou, *Machine Learning*, 1st ed. Springer Singapore, 2021. doi: <https://doi.org/10.1007/978-981-15-1967-3>.
- [7] N. J. Nilsson, “Probabilistic Logic*,” *Artificial Intelligence*, vol. 28, pp. 71–87, 1986, doi: [https://doi.org/10.1016/0004-3702\(86\)90031-7](https://doi.org/10.1016/0004-3702(86)90031-7).
- [8] L. De Raedt, *Logical and Relational Learning*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008. doi: 10.1007/978-3-540-68856-3.
- [9] L. De Raedt and K. Kersting, “Probabilistic Inductive Logic Programming,” in *Algorithmic Learning Theory*, Springer Berlin Heidelberg, 2004, pp. 19–36.
- [10] T. Sato, “A Statistical Learning Method for Logic Programs with Distribution Semantics,” in *Logic Programming: The 12th International Conference*, L. S. Sterling, Ed., The MIT Press, 1995. doi: 10.7551/mitpress/4298.001.0001.
- [11] D. Barber, *Bayesian Reasoning and Machine Learning*. Cambridge: Cambridge University Press, 2012. doi: <https://doi.org/10.1017/CBO9780511804779>.
- [12] R. E. Neapolitan, “Learning Bayesian networks,” in *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Jose, California, USA: Association for Computing Machinery (ACM), Aug. 2007, pp. 1–1. doi: <https://doi.org/10.1145/1281192.1327961>.
- [13] X.-S. Yang, “Mathematical foundations,” in *Introduction to Algorithms for Data Mining and Machine Learning*, Elsevier, 2019, pp. 19–43. doi: 10.1016/b978-0-12-817216-2.00009-0.
- [14] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press, 2009.
- [15] M. Richardson and P. Domingos, “Markov Logic Networks,” *Mach Learn*, vol. 62, no. 1, pp. 107–136, 2006, doi: <https://doi.org/10.1007/s10994-006-5833-1>.
- [16] P. Jackson, *Introduction to Expert Systems*, 3rd ed. Addison-Wesley, 1999.
- [17] S. L. Kendal and M. Creen, *An Introduction to Knowledge Engineering*. Springer-Verlag, 2007.
- [18] S. Russel and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Pearson, 2016. Accessed: Dec. 18, 2024. [Online]. Available: <https://books.google.gr/books?id=XS9CjwEACAAJ>
- [19] J. 'Joyce, “Bayes’ Theorem,” *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, 2021. Accessed: Apr. 30, 2024. [Online]. Available: <https://plato.stanford.edu/archives/fall2021/entries/bayes-theorem/>
- [20] D. L. Poole and A. K. Mackworth, *Artificial Intelligence Foundations of Computational Agents*, Third Edition. Cambridge University Press, 2023.
- [21] A. Farasat, A. Nikolaev, S. N. Srihari, and R. H. Blair, “Probabilistic graphical models in modern social network analysis,” *Soc Netw Anal Min*, vol. 5, no. 1, p. 62, Dec. 2015, doi: 10.1007/s13278-015-0289-6.

- [22] X.-L. Meng and D. Van Dyk, "The EM Algorithm - an Old Folk-song Sung to a Fast New Tune," *J R Stat Soc Series B Stat Methodol*, vol. 59, 1997, Accessed: Dec. 18, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:17461647>
- [23] "Welcome to Python.org." Accessed: Oct. 23, 2024. [Online]. Available: <https://www.python.org/>
- [24] "React." Accessed: Jun. 26, 2024. [Online]. Available: <https://react.dev/>
- [25] "Welcome to Flask — Flask Documentation (3.0.x)." Accessed: Oct. 23, 2024. [Online]. Available: <https://flask.palletsprojects.com/en/3.0.x/>
- [26] H. Asri, H. Mousannif, H. Al Moatassime, and T. Noel, "Using Machine Learning Algorithms for Breast Cancer Risk Prediction and Diagnosis," in *Procedia Computer Science*, Elsevier, 2016, pp. 1064–1069. doi: 10.1016/j.procs.2016.04.224.
- [27] M. O. Adebisi, M. O. Arowolo, M. D. Mshelia, and O. O. Olugbara, "A Linear Discriminant Analysis and Classification Model for Breast Cancer Diagnosis," *Applied Sciences (Switzerland)*, vol. 12, no. 22, Nov. 2022, doi: 10.3390/app122211455.
- [28] M. A. Naji, S. El Filali, K. Aarika, E. H. Benlahmar, R. A. Abdelouhahid, and O. Debauche, "Machine Learning Algorithms for Breast Cancer Prediction and Diagnosis," in *Procedia Computer Science*, Elsevier B.V., 2021, pp. 487–492. doi: 10.1016/j.procs.2021.07.062.
- [29] D. Tomar and S. Agarwal, "Hybrid Feature Selection Based Weighted Least Squares Twin Support Vector Machine Approach for Diagnosing Breast Cancer, Hepatitis, and Diabetes," *Advances in Artificial Neural Systems*, vol. 2015, pp. 1–10, Jan. 2015, doi: 10.1155/2015/265637.
- [30] M. Abdar *et al.*, "A new nested ensemble technique for automated diagnosis of breast cancer," *Pattern Recognit Lett*, vol. 132, pp. 123–131, Apr. 2020, doi: 10.1016/j.patrec.2018.11.004.
- [31] S. Sharma, A. Aggarwal, and T. Choudhury, "Breast Cancer Detection Using Machine Learning Algorithms," in *2018 International Conference on Computational Techniques, Electronics and Mechanical Systems (CTEMS)*, 2018, pp. 114–118. doi: 10.1109/CTEMS.2018.8769187.
- [32] A. E. Kiliç and M. Karakoyun, "Breast Cancer Detection Using Machine Learning Algorithms," *International Journal of Advanced Natural Sciences and Engineering Researches*, vol. 7, no. 3, pp. 91–95, Apr. 2023, doi: <https://doi.org/10.59287/ijanser.401>.
- [33] M. F. Ak, "A comparative analysis of breast cancer detection and diagnosis using data visualization and machine learning applications," *Healthcare (Switzerland)*, vol. 8, no. 2, 2020, doi: 10.3390/healthcare8020111.